

INDIAN INSTITUTE OF TECHNOLOGY, ROPAR



**AUTOMATED DARKNESS
ACTIVATED LAMPS**
(PH102 PROJECT)

SUBMITTED BY:

KHUSH GUPTA (2024EEB1216)

KRISHVIKA (2024EEB1217)

KSHITIJ GUPTA (2024EEB1218)

LAKSHAY GUPTA (2024EEB1219)

LAKSHAY JAIN (2024EEB1220)

AIM :

A hands-on project to build an automatic light system that controls itself based on ambient light using Arduino and a Light Dependent Resistor (LDR).

MATERIAL REQUIRED:

- 1) ARDUINO UNO
- 2) PHOTORESISTOR (LDR)
- 3) 1K RESISTOR
- 4) 5V RELAY MODULE
- 5) BREADBOARD
- 6) JUMPER WIRES
- 7) LIGHT BULB

COMPONENT DESCRIPTION:

1) ARDUINO UNO:

The **Arduino Uno** is a popular microcontroller board used for building electronics projects. It features an easy-to-use interface with 14 digital pins and 6 analog pins, allowing you to connect sensors, motors, and other devices. Powered by an ATmega328P chip, it can be programmed via USB using the Arduino software.

2) RELAY MODULE:

A relay is an electromagnetic switch used to control AC/DC appliances electrically. It works with low voltages, like the 5V output from Arduino pins, and operates by magnetizing a coil to open or close circuits. The high-voltage side has three terminals: Common (COM), Normally Closed (NC), and Normally Open (NO).

3) LDR:

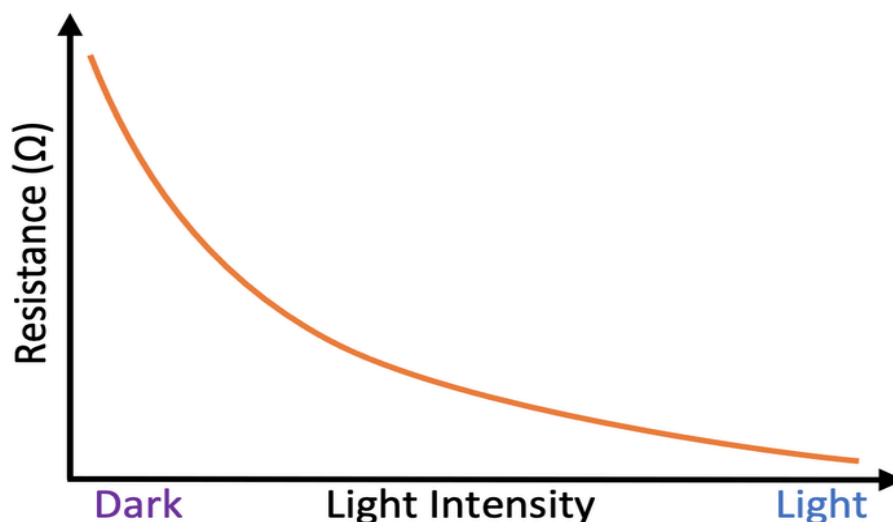
LDR (Light Dependent Resistor) is a variable resistance component that can vary its resistance value with the change in intensity of light and it works on the photoconductivity principle. LDR is made up of high

resistance semiconductor that has resistance in mega ohms in absence of light while several hundred ohms in the presence of light.

THEORY AND WORKING PRINCIPLE:

1) Light Detection with LDR:

- A Light Dependent Resistor (LDR) is used to sense the ambient light intensity. Its resistance decreases as light increases and increases in darkness.
- The LDR is connected in a voltage divider circuit with a fixed resistor which provides an analog voltage signal to the Arduino's analog input pin (e.g., A0).



2) Threshold Comparison in Arduino: The Arduino reads the voltage across the fixed resistor which is directly linked to the light intensity and converts it to a digital value. The Arduino software defines a threshold value that is compared to this value.

In Bright Light:

- The LDR's resistance is low.
- The voltage drop across the LDR is small, so more voltage appears across the fixed resistor.
- The Arduino reads a **higher voltage**.

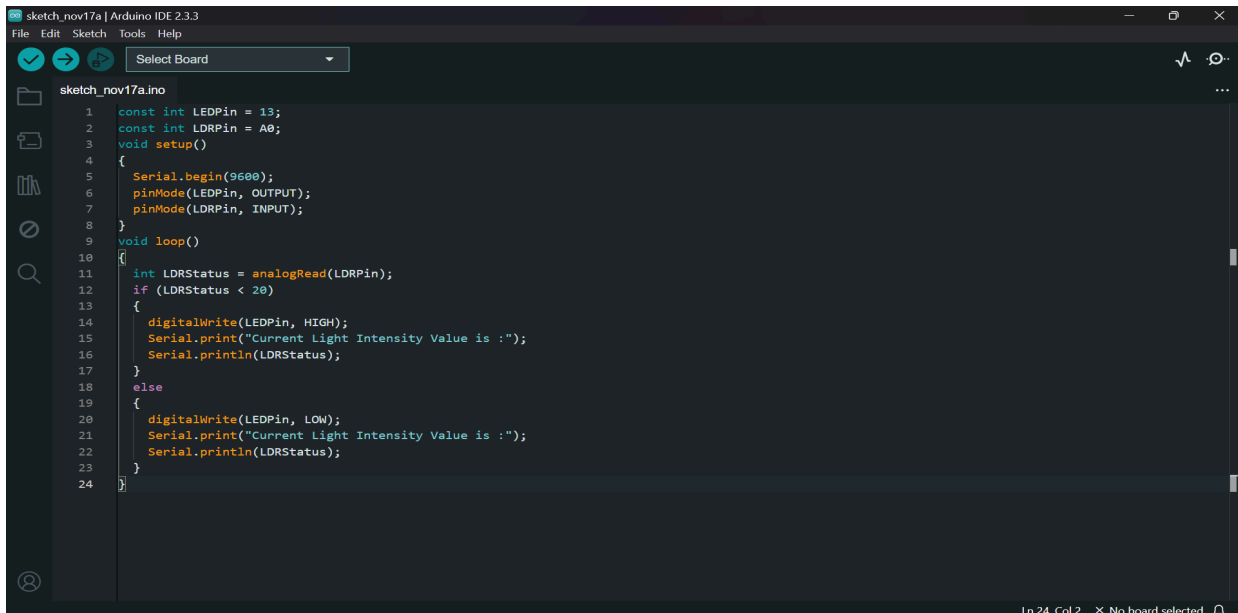
In Darkness:

- The LDR's resistance is high.
- The voltage drop across the LDR is large, so less voltage appears across the fixed resistor.
- The Arduino reads a **lower voltage**.

3)Lamp Control Using Relay:

- The Arduino output is connected to a relay module which acts as a switch.
- When the signal reads less than threshold value, the relay closes the circuit, turning the bulb ON.
- When the signal reads more than threshold value, the relay opens the circuit, turning the bulb OFF.

CODE USED: The following code (written in C++) was used:



```
1  const int LEDPin = 13;
2  const int LDRPin = A0;
3  void setup()
4  {
5      Serial.begin(9600);
6      pinMode(LEDPin, OUTPUT);
7      pinMode(LDRPin, INPUT);
8  }
9  void loop()
10 {
11     int LDRStatus = analogRead(LDRPin);
12     if (LDRStatus < 20)
13     {
14         digitalWrite(LEDPin, HIGH);
15         Serial.print("Current Light Intensity Value is :");
16         Serial.println(LDRStatus);
17     }
18     else
19     {
20         digitalWrite(LEDPin, LOW);
21         Serial.print("Current Light Intensity Value is :");
22         Serial.println(LDRStatus);
23     }
24 }
```

LEDPin: The pin on the Arduino that controls the LED. Pin 13 is typically used for the onboard LED on many Arduino boards, but you can use any digital pin.

LDRPin: The pin where the Light Dependent Resistor (LDR) is connected. In this case, it's connected to analog pin A0.

Serial.begin(9600): Initializes serial communication at a baud rate of 9600. This allows you to send data to your computer and view it in the Serial Monitor.

pinMode(LEDPin, OUTPUT): Configures pin 13 (the LED pin) as an output pin. This means you can control the state of the LED (turn it on or off).

pinMode(LDRPin, INPUT): Configures pin A0 (the LDR pin) as an input pin. The Arduino will read the voltage from the LDR connected to this pin.

analogRead(LDRPin): This function reads the analog voltage on pin A0, where the LDR is connected. The Arduino samples the voltage at the junction of the LDR and the fixed resistor.

$$V_{\text{Output}} = V_{\text{input}} * [R_{\text{fixed}} / (R_{\text{fixed}} + R_{\text{LDR}})]$$

The Arduino converts the input voltage at the analog pin V_{output} into a value between **0 and 1023**.

$$\text{ANALOG VALUE} = V_{\text{Output}} / V_{\text{input}} * 1023$$

if (LDRStatus < 20): This checks whether the light intensity is below a certain threshold (in this case, 20).

- If the value is less than 20, it means the LDR is in a dark environment, so we want to turn on the LED.
- If the value is greater than or equal to 20, the environment is bright enough that we want to turn off the LED.

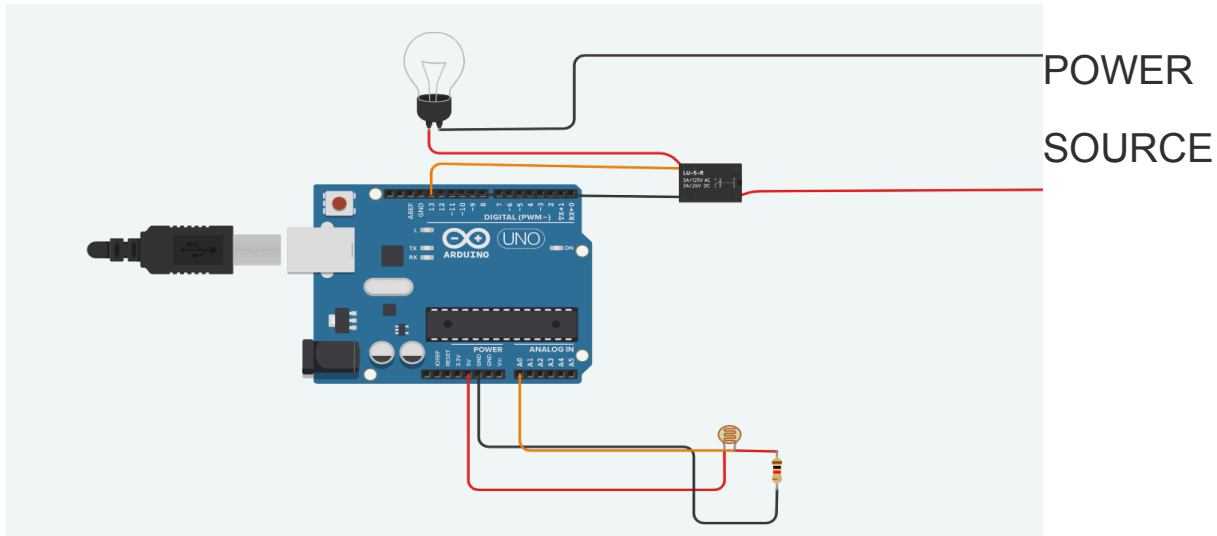
digitalWrite(LEDPin, HIGH): This turns on the LED by sending a HIGH signal to pin 13

digitalWrite(LEDPin, LOW): This turns off the LED by sending a LOW signal to pin 13.

Serial.print("Current Light Intensity value is: "): This prints a label before the value, so you know what the number represents.

Serial.println(LDRStatus): This prints the value read from the LDR (the light intensity) to the Serial Monitor. This helps you monitor the light levels in real-time.

CIRCUIT DIAGRAM:



SOURCES OF ERROR:

1) Sensor Issues:

- **Calibration:** If the light sensor (LDR) is not calibrated properly, the lamp may switch on or off at the incorrect moment.
- **Environmental Factors:** Readings may be impacted by variations in light, such as shadows or reflections.

2) **Insufficient Power:** If the power supply can't provide enough current, the lamp might not work properly.

3) Code Errors:

- **Inaccurate Settings:** It's possible that the code's light threshold differs from the real light levels.
- **Timing Issues:** The lamp may respond slowly due to delays in the code.
- **Logic Errors:** Problems in the code can cause the system to hang or behave wrongly.

4) Component malfunctions:

- **Sensor Wear:** Inaccurate readings may be obtained from sensors over time.
- **Relay Failure:** The parts that regulate the lamp may fail or break.

5) Loose or incorrect wiring can stop the system from working.

APPLICATIONS:

1) Home automation: Automatically turning on lights when it gets dark and off when it's light outside helps save electricity by reducing unnecessary energy consumption.

2) Smart Street Lights: Streetlights that turn on automatically at dusk and off at dawn help improve public safety while conserving energy during daylight hours.

3) Security: Motion-sensing lamps that turn on when it gets dark and detect movement can deter burglars and intruders.

4) Storefronts: Retail stores can use automated lighting to ensure they have the right lighting levels during the evening and nighttime, without manual operation.

5) Solar-Powered Lamps: Automated darkness-activated lamps that run on solar power can give reliable lighting in isolated or off-grid areas without the need for complicated electrical infrastructure.

6) Parking Lots and Public Spaces: Automated lamps can help illuminate parking lots, parks, and public spaces during the night, improving safety and visibility for people.

CONCLUSION:

In conclusion, the development and implementation of an Automatic Night Lamp represent a significant step forward in addressing the need for convenient and energy-efficient lighting solutions, particularly during nighttime hours. Thus we successfully designed and built a project that combines modern technology and user-friendliness to provide ambient lighting when required.

REFERENCES:

[Automatic Night Lamp Using Arduino - Code, Working and Simulation IRJET-V11I449.pdf](#)

CONTRIBUTIONS:

KHUSH GUPTA (2024EEB1216):

- Skillfully tested the project
- Contributed in making the report

KRISHVIKA (2024EEB1217):

- Helped in code and simulation part on Tinkercad
- Contributed in making the report

KSHITIJ GUPTA (2024EEB1218):

- Helped in component selection for the circuit
- Contributed in assembling the circuit

LAKSHAY GUPTA (2024EEB1219):

- Devised the project idea
- Designed the working model and circuit of the project

LAKSHAY JAIN (2024EEB1220):

- Contributed in assembling the circuit